```
    [2] => rigidly
    [3] => defined
    ...
```

There we have it, a pretty simple function to understand. The original string has been chopped up into an array of sub-strings, the separator character being a space character. Note that the separator character is removed from the resulting sub-string, and it's also worth noting that the separator can be a sequence, not just a single byte value (yes, I'm still harping on about byte values; we could have used `explode("\x20", $quote);` to achieve the same results). Take a look at this:

```
print_r(explode("\x61\x6e", $quote));
```

```
Output:
Array
(
    [0] => We dem
    [1] => d rigidly defined areas of doubt
    [2] => d uncertainty!
)
```

The byte sequence `"\x61\x6e"` is simply the ASCII letters `'an'`, and we can see that the string has been split at every occurrence of that sequence (and those characters are not present in the resulting data). `explode()`, like most other string functions, is binary safe:

```
$weirdString = "So long, and thanks\x00for all the fish.";

print_r(explode("\x00", $weirdString));
```

```
Output:
Array
(
    [0] => So long, and thanks
    [1] => for all the fish.
)
```

str_split(), mb_str_split()